Assignment 3
Bridge Pattern

Name: Alikhan Smagul

Group: SE-2529

Topic: A

Repository: https://github.com/alihzzz/SDP_Assignment3.git

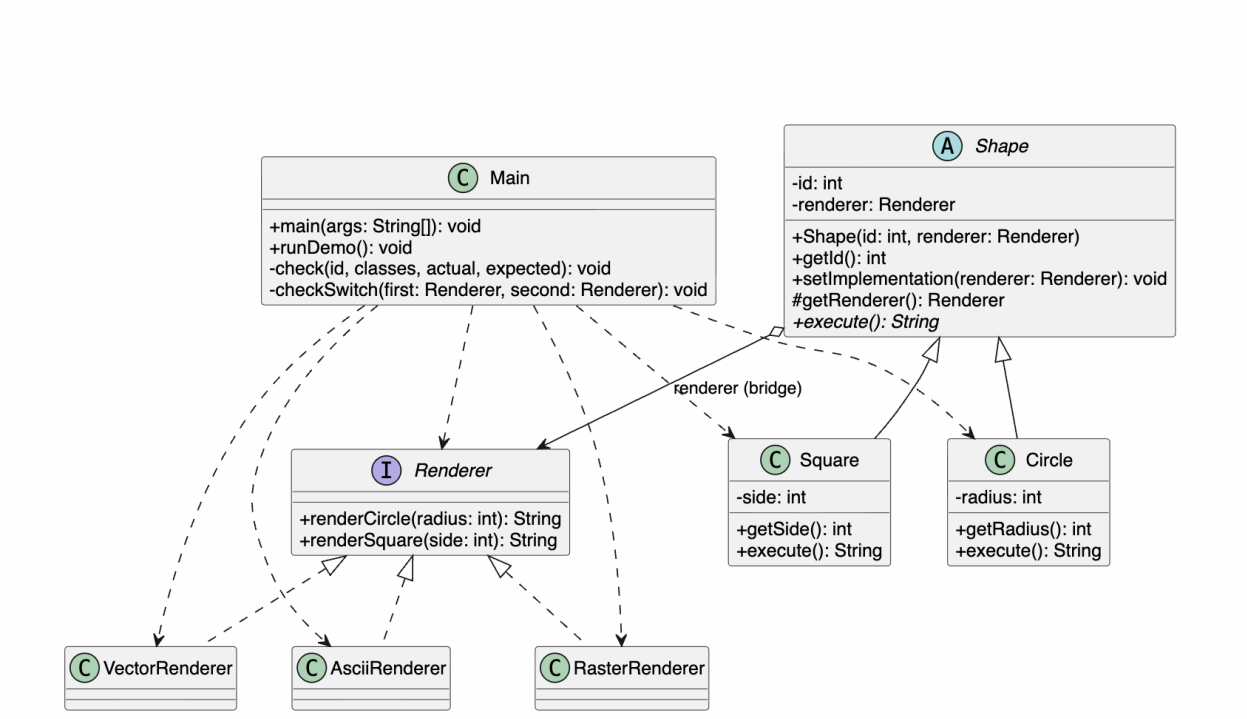
Submitted commit: git rev-parse 3a005a1

Base commit: git rev-parse 9d0855b

1. Two dimensions

The application has two dimensions that change independently. The first is what is drawn: the abstraction Shape with subclasses Circle and Square. The second is how it is drawn: the implementor interface Renderer with implementations VectorRenderer, RasterRenderer and AsciiRenderer. Without Bridge, every combination would need its own class (for example VectorCircle, RasterSquare), and each new shape or renderer would multiply the number of classes. With Bridge, Shape holds a reference to the Renderer interface, so both hierarchies grow separately and are connected by composition.

2. UML class diagram



3. Clean Code excerpts

a) Separation of responsibilities.

```
return getRenderer().renderCircle(radius);
```

Circle only knows what to draw and its size. How to draw is the responsibility of Renderer. Benefit: a change in rendering does not touch the shapes.

b) Meaningful names.

```
public void setImplementation(Renderer renderer)
```

```
boolean stateUnchanged = ...
```

Names describe the purpose, so the code can be read without comments. Benefit: easier to understand and maintain.

c) Small focused methods.

```
public String execute() { return getRenderer().renderSquare(side); }
```

Each method does one thing. Benefit: easy to test and explain.

d) Avoiding duplication.

```
private static void check(String id, String classes, String actual, String expected)
```

One method compares the actual and expected result and prints PASS or FAIL for all checks. Benefit: no repeated if/else blocks, and the result is calculated, not hard-coded.

e) Encapsulation of state.

```
private final int id;

private Renderer renderer;
```

Fields are private; the renderer can be changed only through setImplementation. Benefit: the state is controlled and cannot be broken from outside.

4. How I3 was added

I added only a new class AsciiRenderer that implements Renderer, and updated Main to run checks T6 and T7. Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer were not changed; this is shown in extension.diff. One trade-off of Bridge is extra complexity: an additional interface and delegation make a small program larger and less direct.

5. Bridge vs Adapter

Bridge is designed in advance so that the abstraction and the implementation can vary independently. Adapter is used after the fact to make an existing class with an incompatible interface work with the interface the client expects. In my application the renderers were written for the Renderer interface from the start, so this is Bridge. An Adapter would be needed, for example, to connect an external drawing library with different method names.

6. Sources

Lecture 4.